

# 红黑树

---

说起红黑树，我们不得不从二叉树开始说起。

在学二叉树的时候，我们知道，二叉树是指每个结点最多只有两个子结点的树。

二叉树，是指树中每个结点最多只有两个结点的树。当然，二叉树本身好像没有什么太大的作用。我们平时所说的二叉树，基本上就是指二叉排序树。

## 二叉查找树（BST）

---

二叉查找树就是在二叉树的基础上增加有序性，这个有序性一般是指自然顺序，有了有序性，我们就可以使用二叉树来快速的查找、删除、插入元素了。

但是，二叉查找树有个非常严重的问题，试想，还是这三个元素，如果按照A、B、C的顺序插入元素会怎样？

那么如果按照顺序插入，就会变成一个单链表。没错，当按照元素的自然顺序插入元素的时候，二叉查找树就退化成单链表了，单链表的插入、删除、查找元素的时间复杂度是多少？

所以，在极限状态下，二叉查找树的时间复杂度是非常差的。

既然，插入元素后有可能导致二叉查找树的性能变差，我们是不是增加了一些手段，让插入后的二叉查找树依然性能良好？

这种手段，就叫做平衡。可以做到自平衡的树就叫做平衡树。

## 平衡树

---

平衡树，是指插入、删除元素后可以自平衡的二叉查找树。我们平衡的手段，就是旋转。

平衡这个概念一直都有，直到62年，发明了第一种平衡树---AVL树。

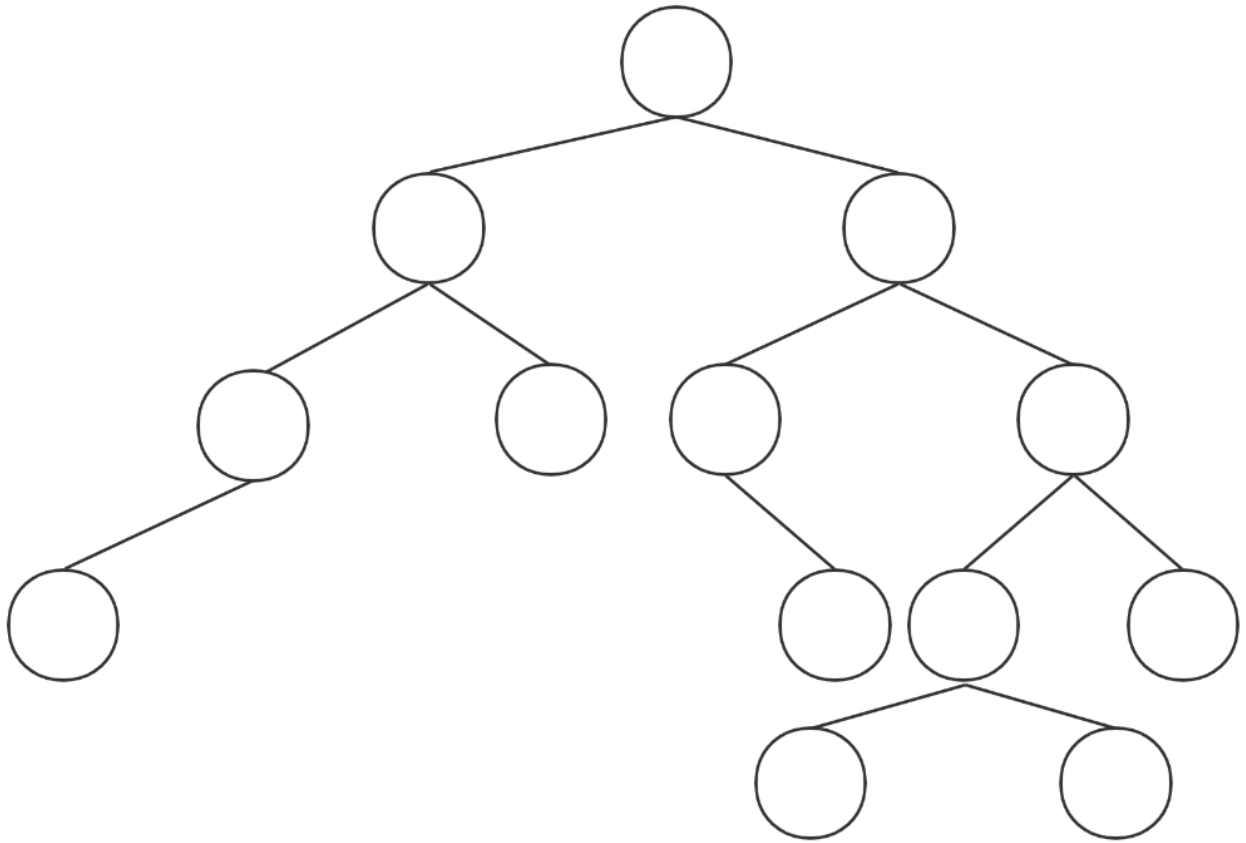
严格来说，平衡树是指可以自平衡的二叉查找树，关键词就是：自平衡、二叉、查找（有序）。

## AVL树

---

AVL树是指任意节点的两个子树的高度差不超过1的平衡树。

当数据量非常多的时候，你会非常难以判断这是否是一颗AVL树，比如



如果把上面看成一颗二叉排序树，他是一颗AVL树，其实你很难一眼就看出来他是一颗AVL树，这就是AVL树的第一个缺点，不够直观，特别是当结点特别多的时候。

第二个缺点就是，在插入和删除的时候自平衡的过程非常的复杂。

基于这些缺点，所以，后来又发展出来了各种各样的神奇的平衡树。

## 多路平衡二叉树

### 索引

一般来说，我们操作的数据都是存储在内存（CPU）中的，但如若我们要操作的数据集非常大，大到内存已经没办法处理了怎么办呢？如数据库中的上千万条记录的数据表、硬盘中的上万个文件等，他们必然不能都存储在内存中，而是存储在外存中的。

对于外存中的数据，常见如数据库，我们通常通过索引表来进行数据查找和交互，一般来说，索引表本身也很大，因为数据库的数据非常多，因此索引也不可能全部存储在内存中，因此索引表往往也是以索引文件的形式存储的磁盘上。Mysql的MyISAM引擎的索引文件和数据文件是分离的，一张数据库表就有它对应的索引表，索引表中一个索引对应一条数据库记录的磁盘地址，内存中发起请求通过指定索引的查找索引表即可定位唯一的一条数据；当然Mysql的InnoDB引擎的索引表也是数据表，即索引表保存了索引和完整的数据记录。但是不管怎么说数据的查找都会依赖索引，并且建议通过索引查找，因为如果没有走索引，那就会走全表扫描，使得查找效率大大降低。因为索引文件同样存储在磁盘上，这样的话，索引查找过程中每查找一次就要产生一次磁盘I/O消耗，相对于CPU存取，I/O存取的消耗要高几个数量级，访问磁盘的成本大概是访问内存的十万倍左右。

实际上，考虑到磁盘IO是非常高昂的操作，计算机操作已经系统做了一些优化，当一次IO时，不光把当前磁盘地址的数据，而是把相邻的数据也都读取到内存缓冲区内，局部预读性原理告诉我们，当计算机访问一个地址的数据的时候，与其相邻的数据也会很快被访问到。

每一次IO读取的数据我们称之为`一页`(page)。具体一页有多大数据跟操作系统有关，一般为4k或8k大小的连续磁盘块，也就是我们读取一页内的数据时候，实际上才发生了一次IO，这个理论对于索引的数据结构设计非常有帮助，通常，索引节点的大小被设计为一页的大小。

## 多路平衡查找树

对于一旦涉及到这样的外部存储设备（外存），关于时间复杂度的计算就会发生变化，访问某个表/集合元素的时间已经不仅仅是寻找该元素所需比较次数的函数，我们必须考虑对硬盘IO进行操作的时间。由于IO耗时远大于CPU耗时，所以此处评价一个数据结构作为索引的优劣最重要的指标就是要尽量减少查找过程中磁盘IO的存取次数。

我们之前谈的树，都是一个节点可以有多个孩子，但是它自身只存储一个元素。二叉树限制更多，节点最多只能有两个孩子。一个节点只能存储一个元素，在元素非常多的时候，就使得要么树的度非常大（节点拥有子树的个数的最大值），要么树的高度非常大，甚至两者都必须足够大才行，这就使得IO次数非常多，这显然成了时间效率上的瓶颈，并且由于一次IO读取一个节点的数据，普通二叉树并不能容纳更多的数据，这样又造成了磁盘块空间的浪费。

以上种种限制迫使我们设计出每一个节点可以存储多个元素，并且数据结构的高度可控的数据结构，为此引入了多路查找树的概念。一颗平衡多路查找树同样可以使得数据的查找效率保证在 $O(\log N)$ 这样的对数级别上，此时底数为叉数或者阶。

多路查找树（multl-way search tree），其每一个节点的孩子数可以多于两个，且每一个节点处可以存储多个元素。由于它是一颗平衡查找树，所有元素之间存在某种特定的排序关系。在这里，每一个节点可以存储多少个元素，以及它的孩子数的多少是非常关键的。

## 2-3树

---

因为AVL树所带来的搜索性能的提升，不足以弥补平衡树所带来的性能损耗。所以，就开始思考，有没有一种绝对平衡的树。没有高度差，没有高度差就没有平衡因子，没有平衡因子就没有旋转操作。

随着这种思考，衍生出了2-3树。也就是二叉-三叉树。

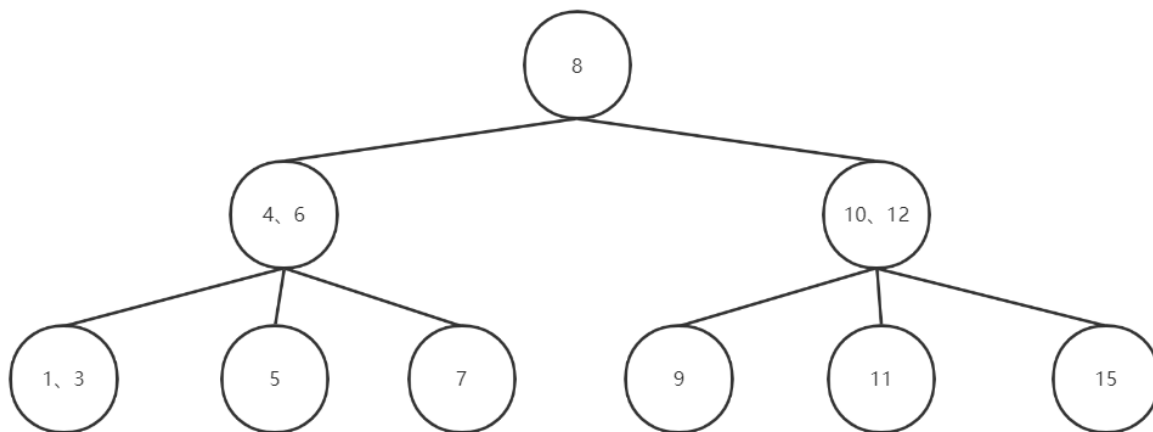
2-3树就是一种绝对平衡的树，任意节点到它所有的叶子结点的深度都是相等的。

## 定义

一颗2-3树或为一颗空树，或有以下结点组成：

2-节点：含有一个元素和两个子树，左子树的所有元素的值均小于它的父结点，右子树所有元素的值均大于它的父结点。

3-节点：还有两个元素和三个子树（左中右子树），左子树所有元素的值均小于它的父结点，中子树所有元素的值都位于父结点两个元素之间，右子树所有元素的值均大于它的父结点。



## 2-3树查找元素

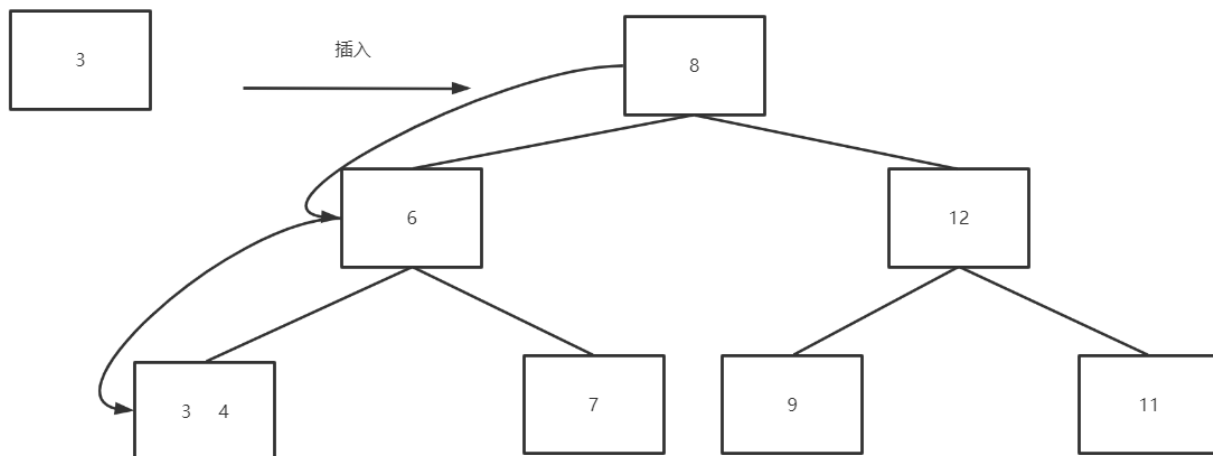
2-3树的查找类似于二分，根据元素的大小来决定来查找的方向。要判断一个元素是否存在，我们就要先将待查找元素和根元素比较，如果他和任意一个相等，那查找命中，否则根据比较结果来选择查找方向。

## 2-3树插入元素

插入元素首先进行查找命中。若查找命中则不插此元素。如果支持重复的元素则将这个元素对象添加一个属性count。若查找未命中，则在叶子结点中插入这个元素。

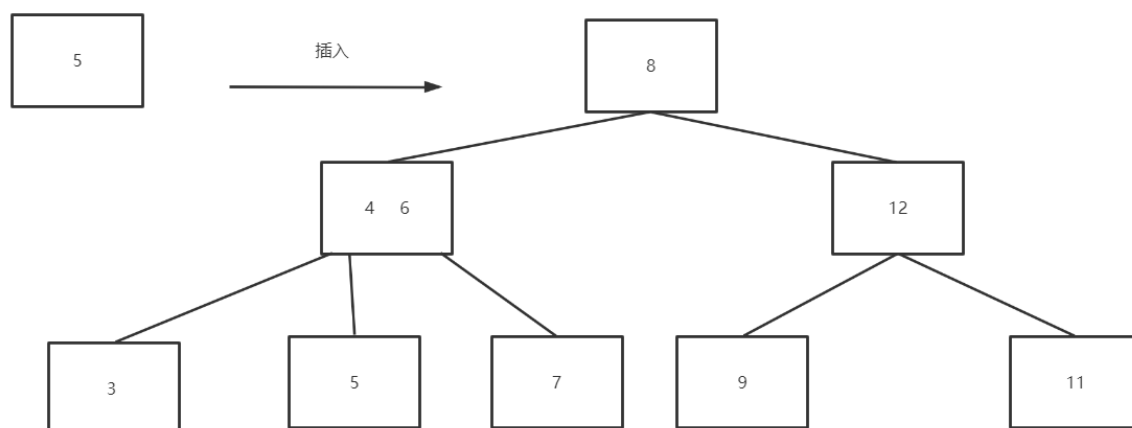
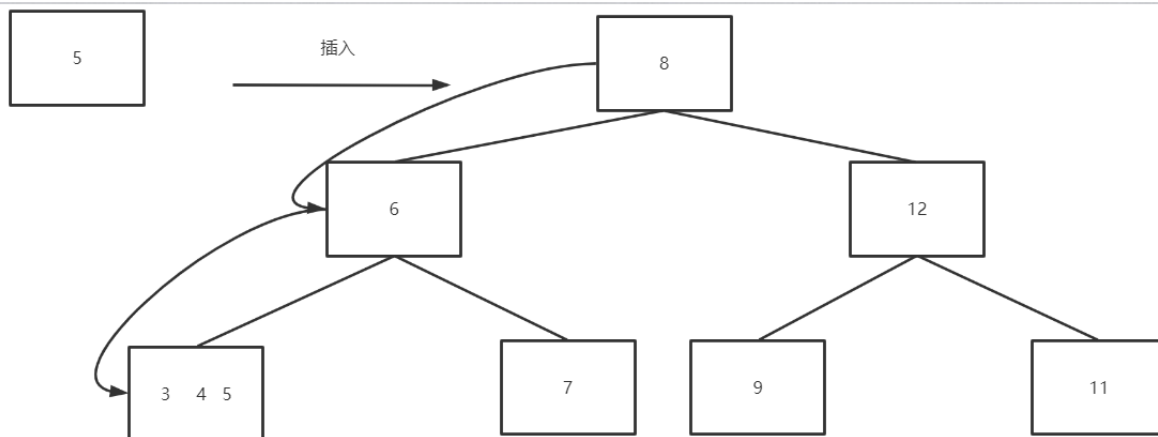
空树的插入很简单，创建一个结点就可以了。如果不是空树，插入又分成了四种情况：

### 1、向2-结点中插入元素



### 2、向一颗只含有一个3-节点的树中插入元素

如果命中查找结束于3-节点，先临时将其成为4-节点，把待插入元素添加到其中，然后将4-节点转化为3个2-节点，中间的节点成为左右节点的父节点。如果之前临时4-节点有父节点，就会变成向一个父节点为2-节点的3-节点中插入元素，中间节点与父节点为2-节点的合并。

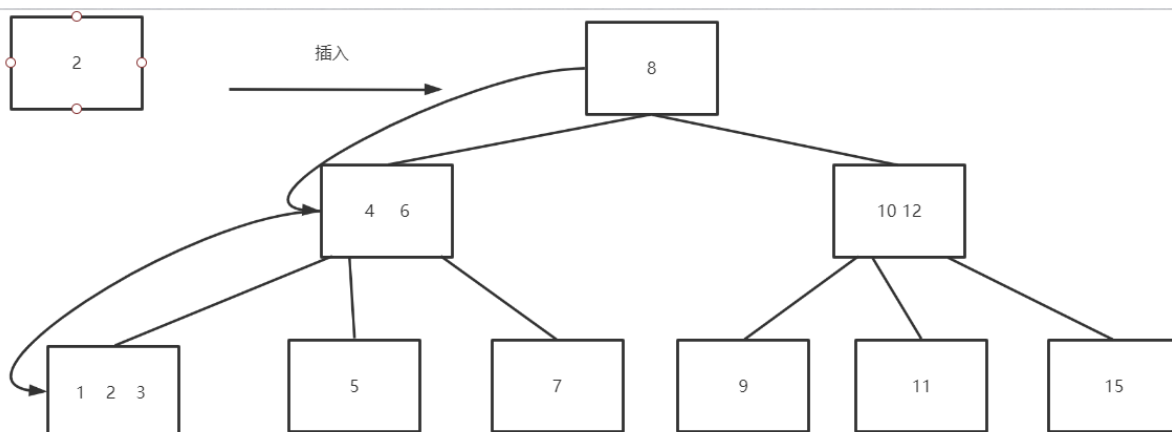


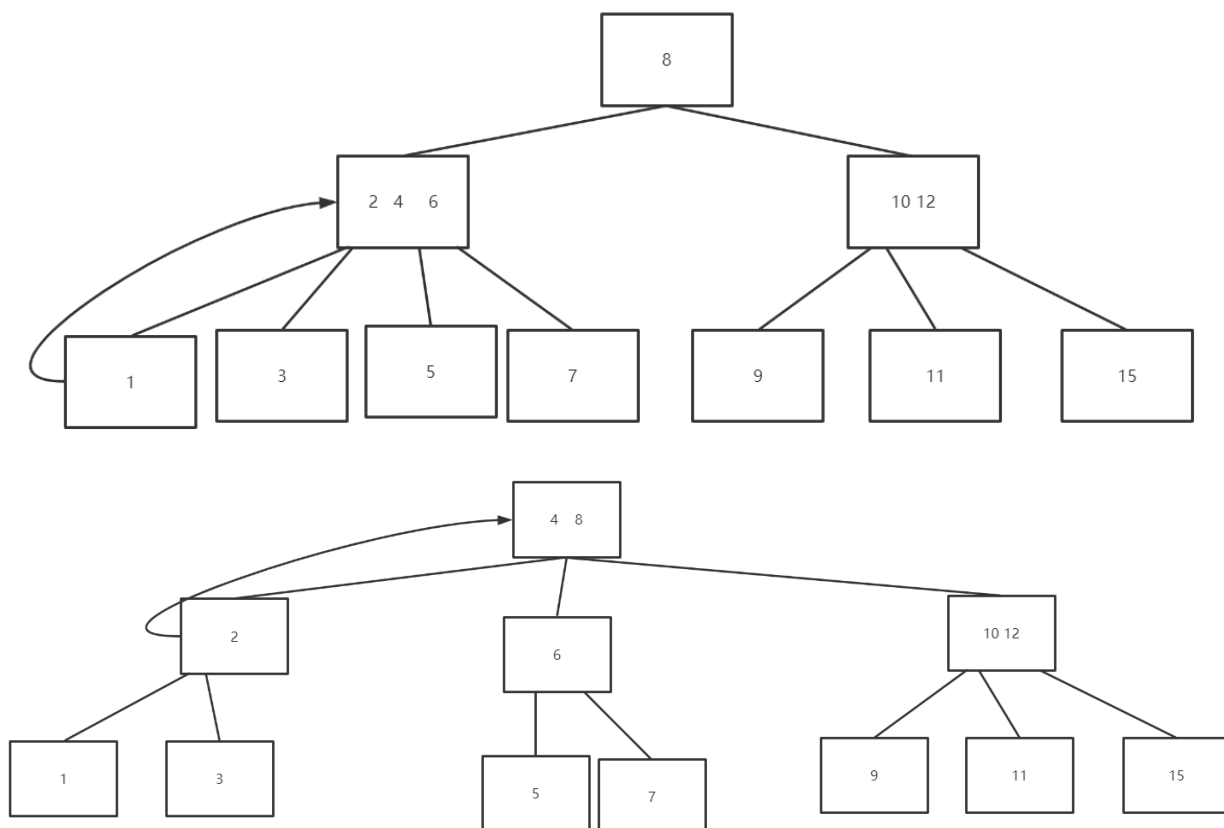
### 3、向一个父结点为2-节点的3-节点中插入元素

同前者

### 4、向一个父结点为3-节点的3-节点中插入元素

插入元素后一直向上分解临时的4-节点，直到遇到2-节点的父节点变成3-节点不再分解。如果达到树根节点还是4-节点，则进行分解根节点，此时树高+1（只有分解根节点才会增加树高），下面动画2-3树插入会出这个例子。





## 2-3树的删除操作

2-3树的删除也分为三种情况，与插入相反。

### 1、当删除元素为3-节点的叶子结点上

只需要删除该元素即可，不会影响到整棵树的其他结点结构。

### 2、当删除元素位于非叶子结点

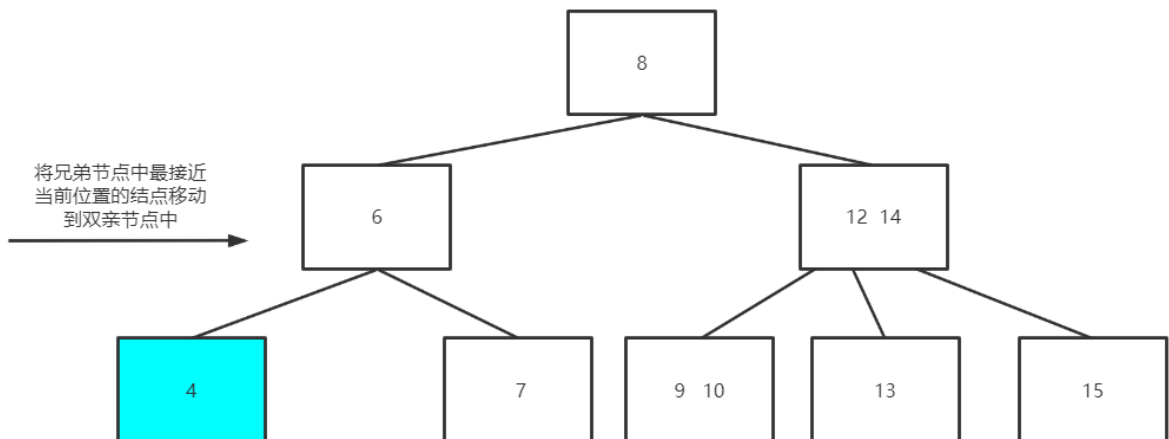
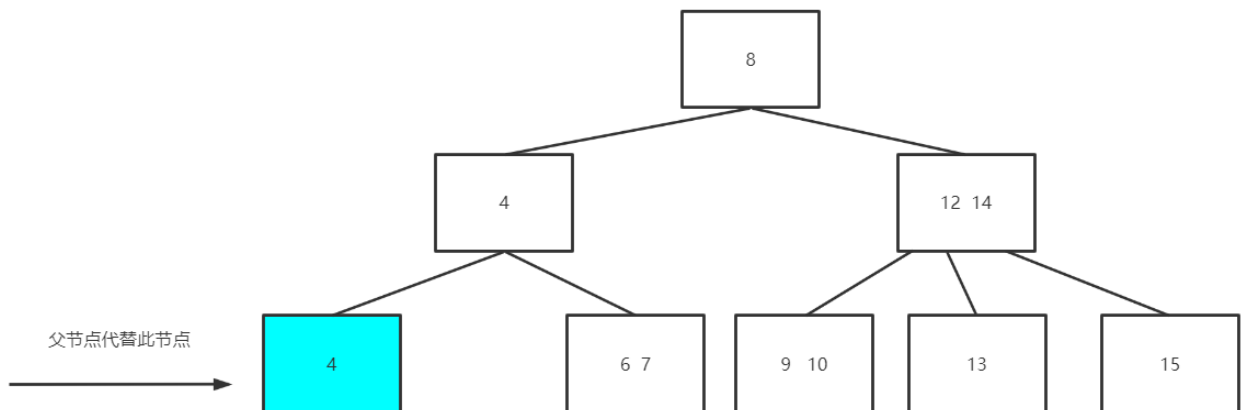
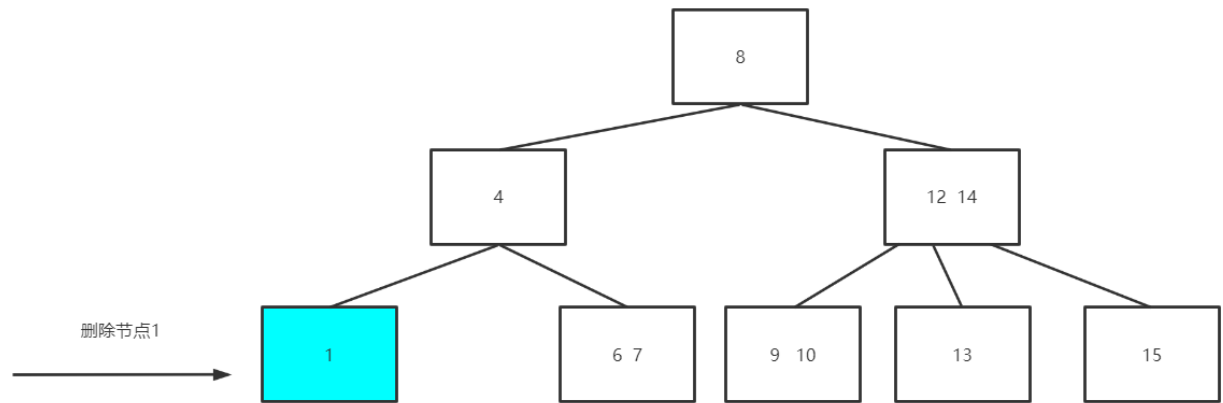
使用中序遍历找到待删除节点的后继节点，然后将后继节点与待删除节点位置互换，此时就将问题转化为删除节点为叶子节点（平衡树的非叶子节点中序遍历后继节点肯定是叶子节点），如果该叶子是3-节点，则跟情况（1）一样，如果该节点是2-节点，则跟后面的情况（3）一样；

### 3、当删除元素位于2-结点的叶子结点上

删除元素2-结点的叶子结点的步骤相对很复杂，删除后需要做出相应的判断。并根据判断结果调整树的结构。

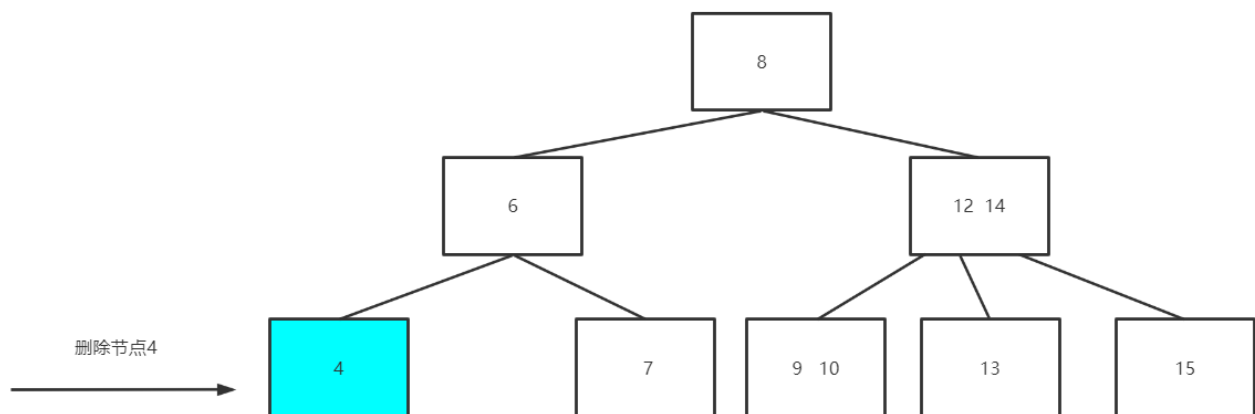
#### 1、删除结点为2结点，父结点为2结点，兄弟结点为3结点。

操作步骤：当前待删除节点的父节点是2-节点、兄弟节点是3-节点，将父节点移动到当前待删除节点位置，再将兄弟节点中最接近当前位置的key移动到父节点中。

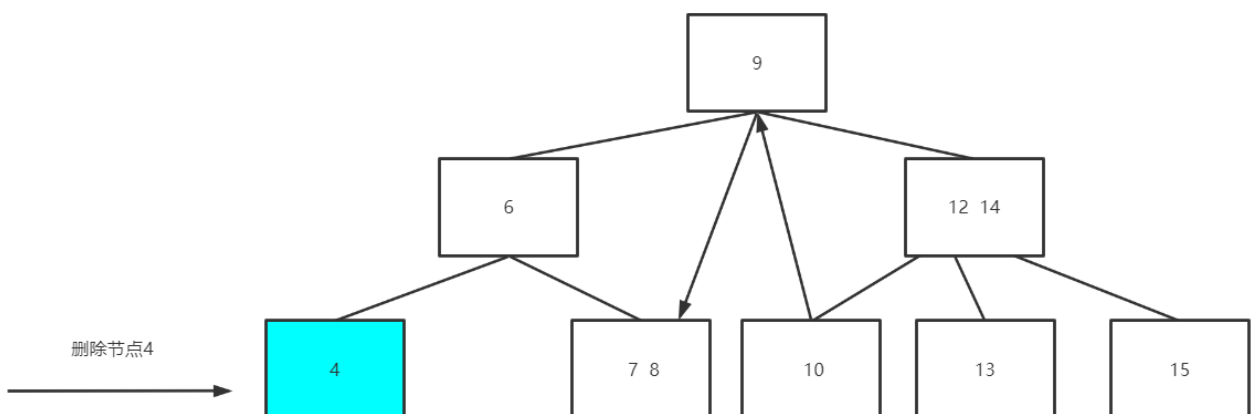


## 2、删除结点为2-结点，父结点为2-结点，兄弟结点为2-结点

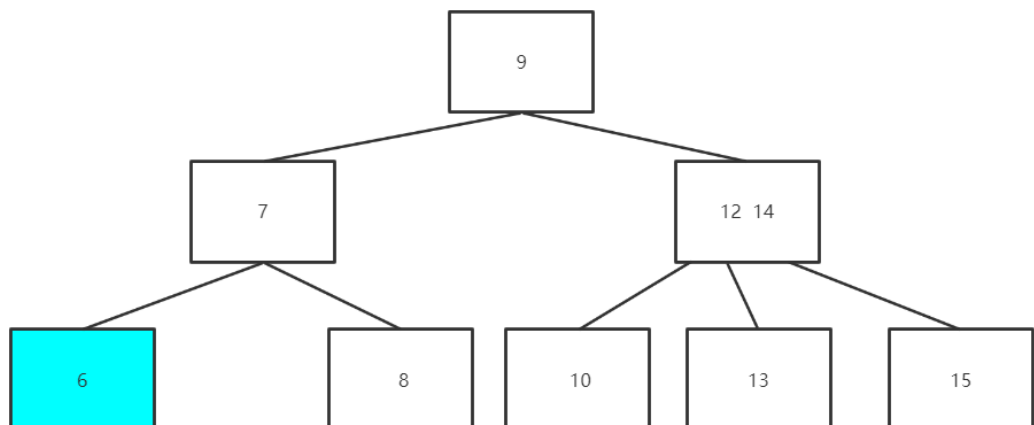
操作步骤：当前待删除节点的父节点是2-节点、兄弟节点也是2-节点，先通过移动兄弟节点的中序遍历直接后驱到兄弟节点，以使兄弟节点变为3-节点；



删除结点4为2-结点，兄弟结点7也为2结点，需要中序遍历得到兄弟结点7的直接后继8，然后结点7和8构成3-结点。

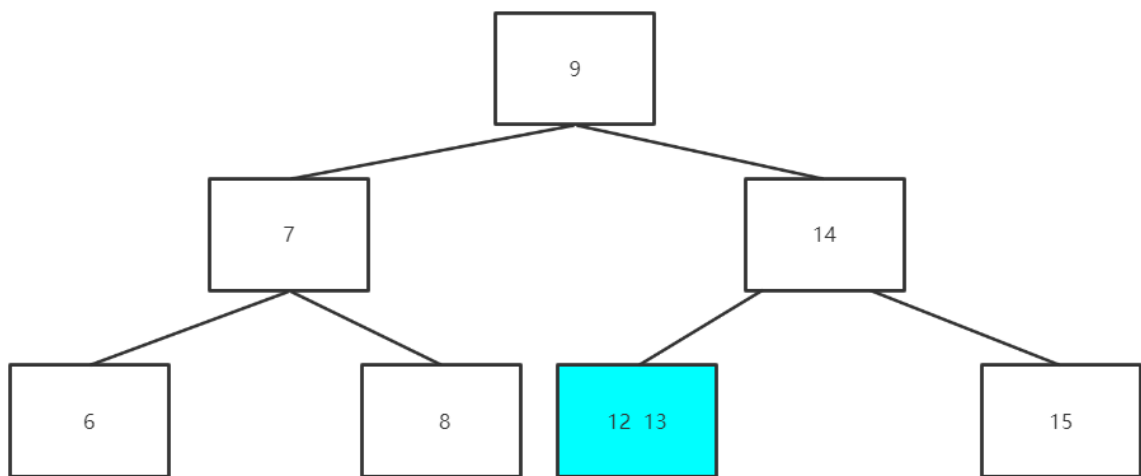
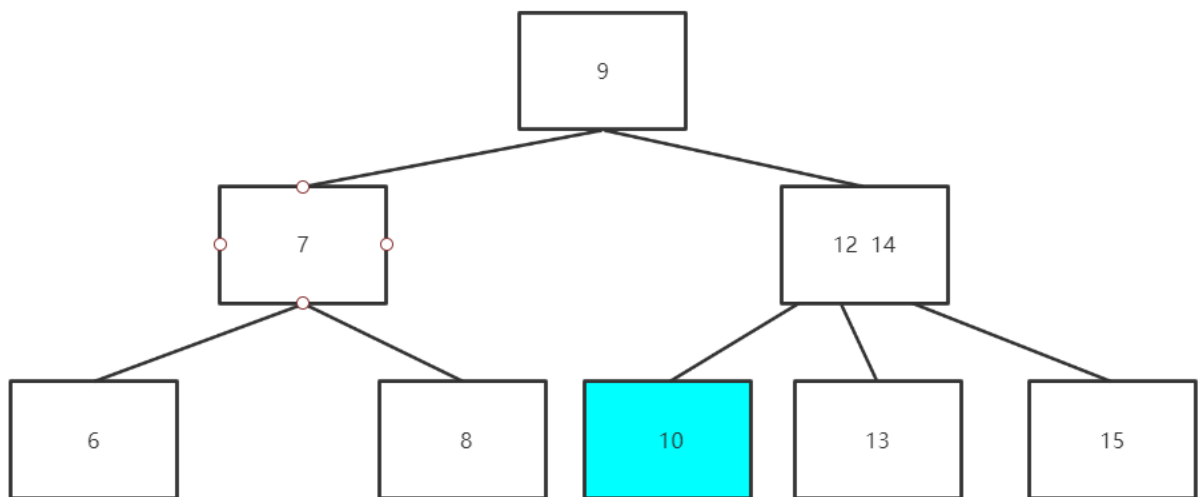


重复1情况



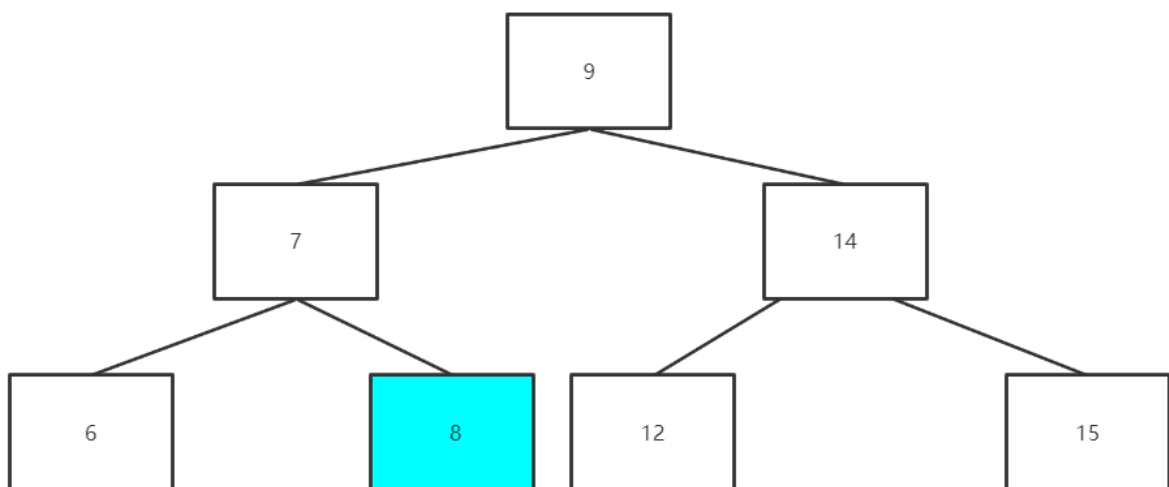
### 3、删除结点为2-结点，父结点为3-结点

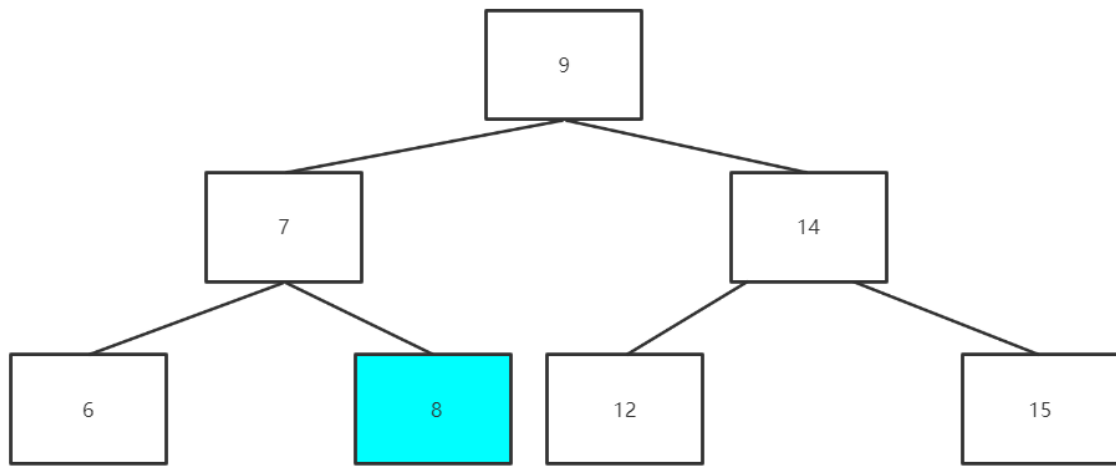
操作步骤：当前待删除节点的父节点是3-节点，拆分父节点使其成为2-节点，再将再将父节点中最接近的一个拆分key与中孩子合并，将合并后的节点作为当前节点。



删除节点10，并将父节点最为接近10结点的数字下移，然后和节点13构成3-结点

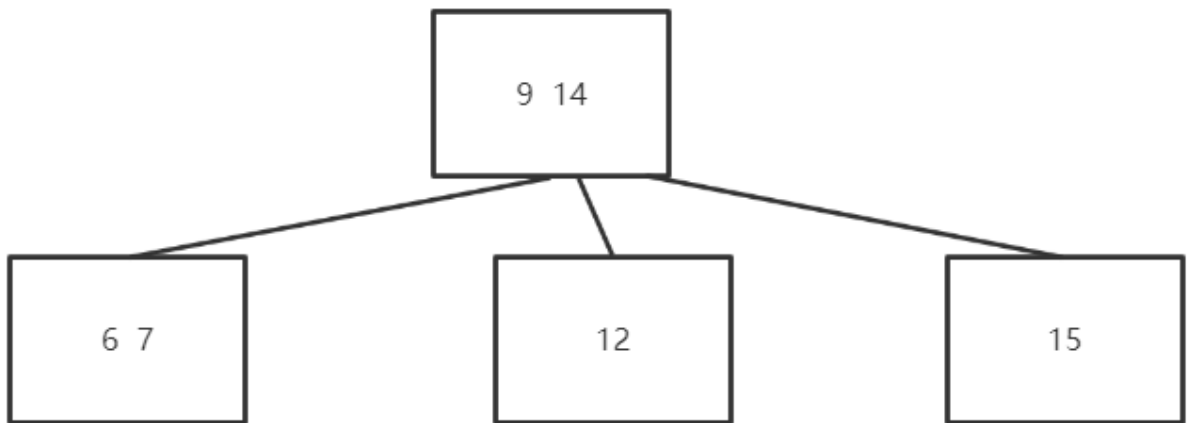
2-3树为满二叉树，删除叶子结点





2-3树如果变成一颗满二叉树，将2-3树的层数减少，将兄弟结点6上移，同时将父节点7所有的兄弟节点上移志父节点9中

---



2-3 树作为一种平衡查找树，查询效率比普通的二叉排序树要稳定许多。但是2-3树需要维护两种不同类型的结点，查找和插入操作的实现需要大量的代码，而且它们所产生的额外开销可能会使算法比标准的二叉查找树更慢。

可以看到，上面自平衡的过程中，出现了一种节点，它具有四个子节点和三个数据元素，这个节点可以称作4节点，如果把4节点当作是可以允许存在的，那么，就出现了另一种树：2-3-4树。

## 2-3-4树

---

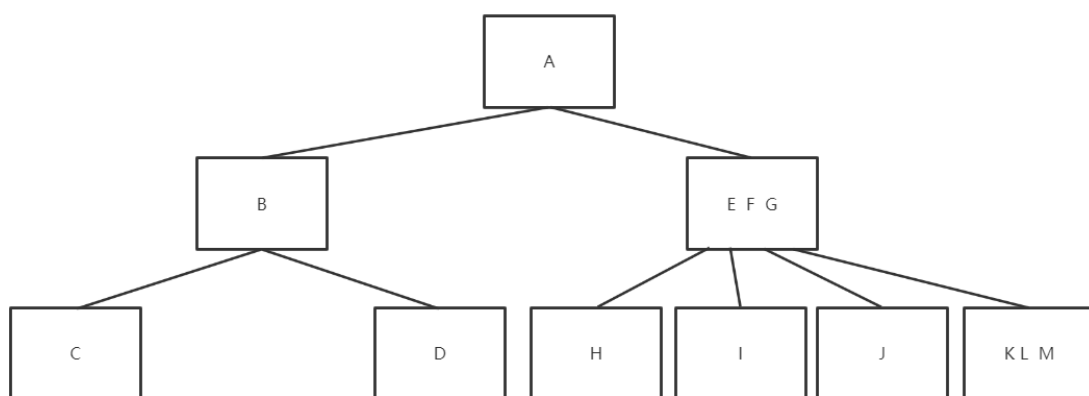
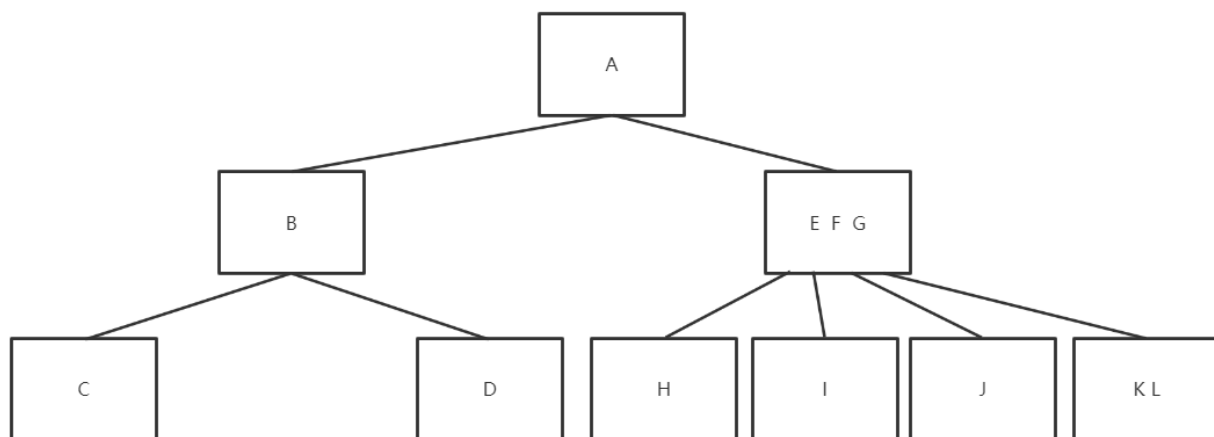
2-3-4树，它的每个非叶子节点，要么是2节点，要么是3节点，要么是4节点，且可以自平衡，所以称作2-3-4树。

2节点、3节点、4节点的定义在上面已经提及，我们再重申一下：

2节点：包含两个子节点和一个数据元素；

3节点：包含三个子节点和两个数据元素；

4节点：包含四个子节点和三个数据元素；

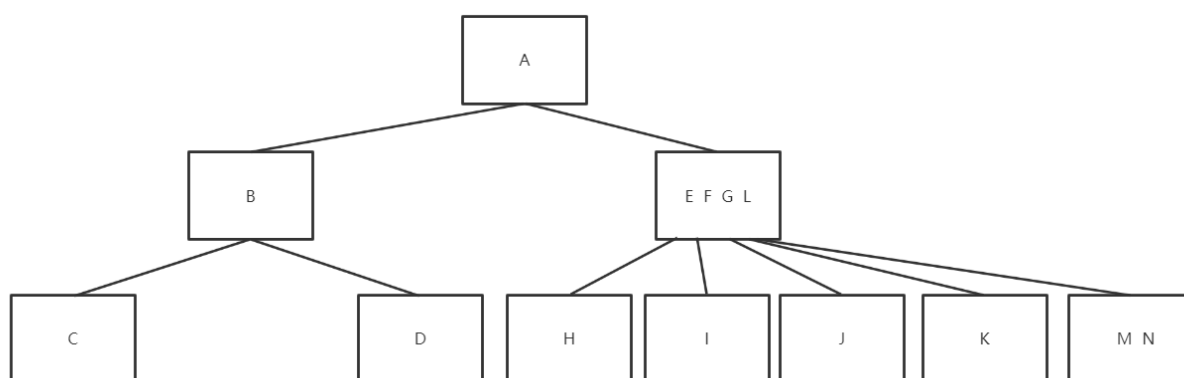


插入M

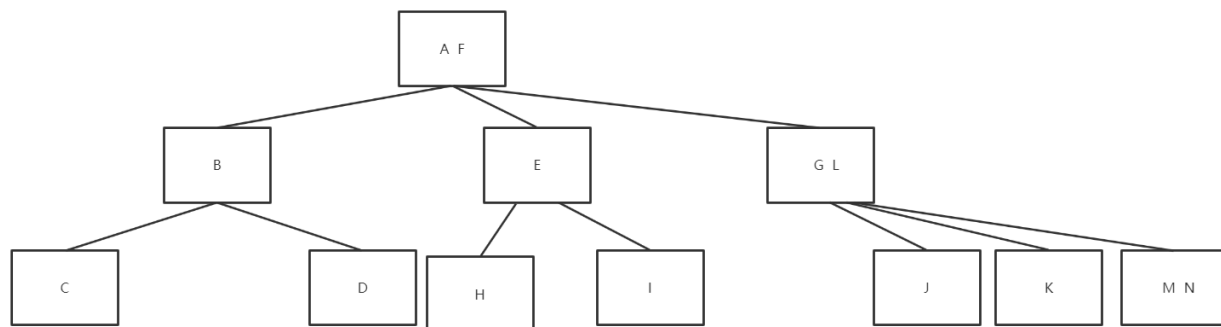


插入M，依旧符合2-3-4树的规则。在插入N呢？

插入N，L上移。



H上移



是不是挺简单的，至少比AVL树那种左旋右旋简单得多。同样地，在2-3-4树自平衡的过程中出现了临时的5节点，所以，如果允许5节点的存在呢？嗯，2-3-4-5树由此诞生！同样地，还有2-3-4-5-6树、2-3-4-5-6-7树……子子子孙，无穷尽也~所以，有人就把这一类树归纳为一个新的名字：B树。

## B树

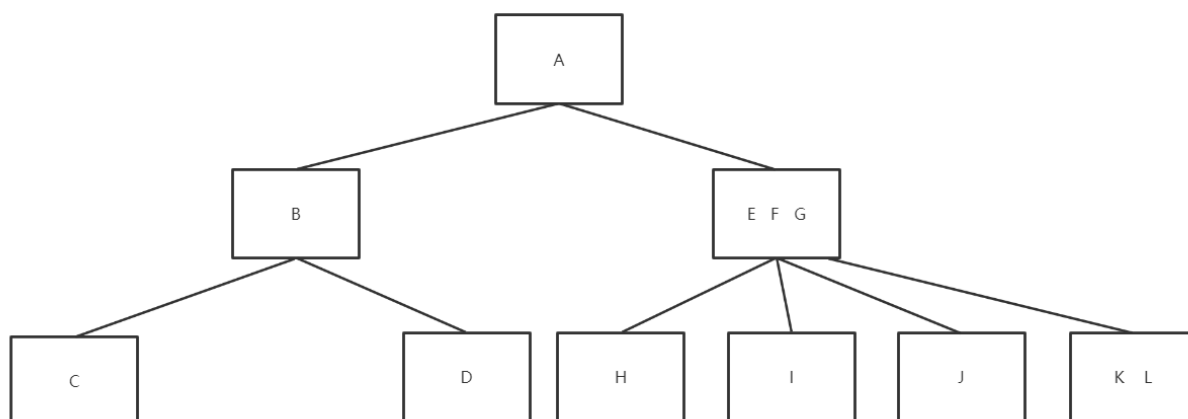
B树，表示的是一类树，它允许一个节点可以有多个子节点，同时，也是自平衡的，叶子节点的高度都是相同的。所以，为了更好地区分一颗B树到底属于哪一类树，我们给它一个新的属性：度（Degree）。具有度为3的B树，表示一个节点最多有三个子节点，也就是2-3树的定义。具有度为4的B树，表示一个节点最多有四个子节点，也就是2-3-4树的定义。

B树，一个节点可以存储多个元素，有利于缓存磁盘数据，整体的时间复杂度趋向于 $O(\log n)$ ，原理也比较简单，所以，经常用于数据库的索引，包括早期的mysql也是使用B树来作为索引的。但是，B树有个大缺陷，比如，我要按范围查找元素，以上面的2-3-4树为例，查找大于B且小于K的所有元素，该怎么实现呢？很难，几乎无解，所以，后面又出现替代B树的方案：B+树。当然了，B+树不是本节的重点，本节的重点是红黑树。

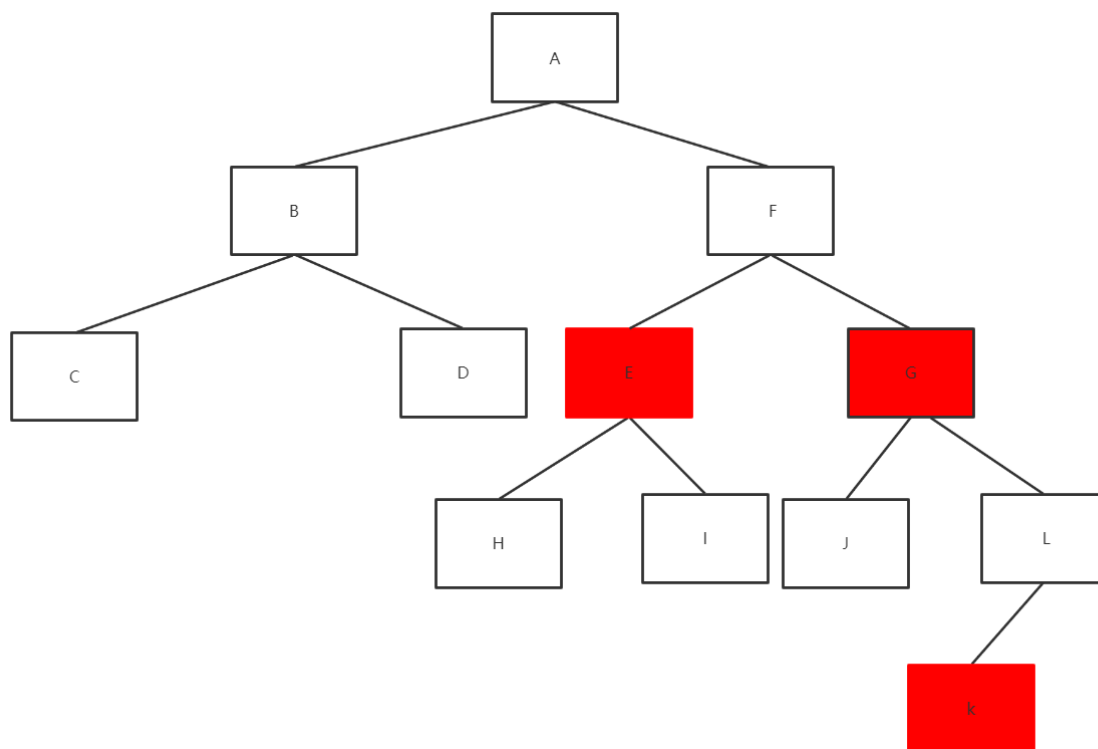
来了来了，有意思的红黑树来了~~

## 红黑树

我们先用图来体会



将3结点和4结点拆分 并且用红线来连接



我可以跟大家说，这棵树，就是一颗红黑树。红黑树就是2-3-4树。

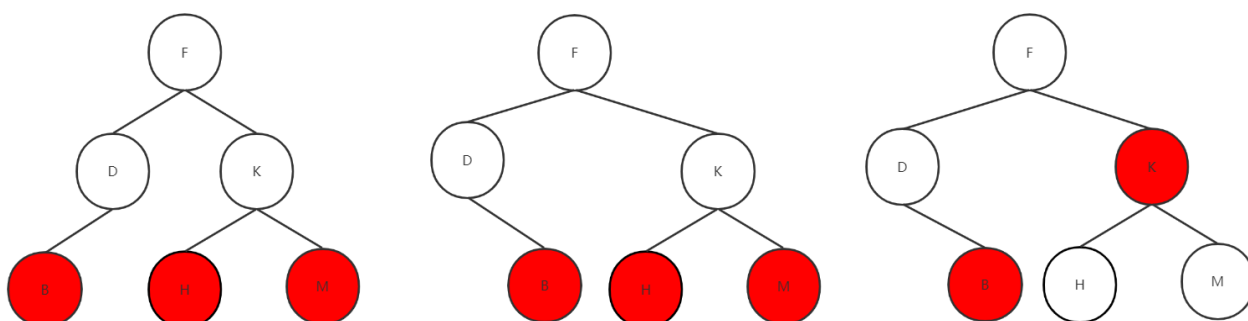
我们知道2-3-4的插入、删除、查找元素的原理是相当简单的，那么，我们是不是可以利用2-3-4树来记忆红黑树呢？答案是肯定的，我们就来看看如何利用2-3-4树来快速掌握红黑树，再也不用死记硬背了~~

## 左倾红黑树、右倾红黑树、AA树

那么，到底什么是红黑树呢？红黑树就是，每个结点都是带有颜色的二叉查找树，颜色或者是红色或者是黑色。

首先：红黑树是一颗二叉查找树，另外，他必须满足五点要求：

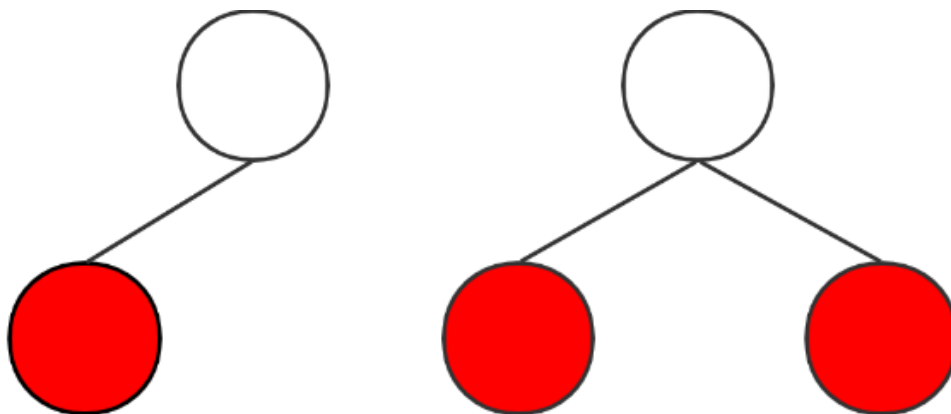
- 1、节点是红色或者黑色
- 2、根结点是黑色
- 3、所有的叶子结点是黑色（叶子结点是NULL结点）
- 4、每个红色结点的两个子结点都是黑色（从根结点到每个叶子结点的路径不能有两个连续的红结点）
- 5、从任何一个结点到每个叶子结点的所有路径都包含相同数目的黑色结点



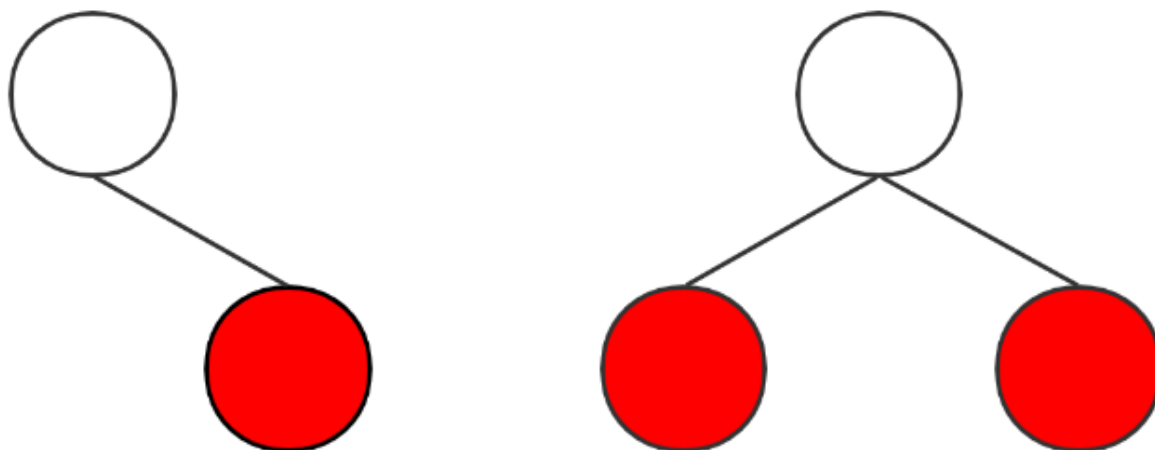
这个时候可能有的人说，不是啊，因为叶子结点有的是红色的呀。注意，红黑树中的叶子结点，一定是null结点。所以，让我们补齐叶子结点来看。所以，我们随便画一颗树，都可能满足红黑树的定义。因此，其实我们随便画一颗树，都可能满足红黑树的定义。为了方便记忆，我们将红黑树分成：左倾红黑树、右倾红黑树、AA树。

一个节点如果有红色子节点，那么，它的红色子节点是向左倾斜的。怎么理解呢？我们还是把上面的null节点干掉哈，叶子节点都是null节点，那是经典红黑树的讲法。我们来看，一个节点要么有一个子节点，要么有两个子节点，对吧。如果这个节点有红色的子节点呢，也是一个或者两个，如果只有一个红色子节点的话，那么，这个子节点只能在左边，如果是两个红色子节点，那就不用管。

所以，整棵红黑树中，如果存在红色节点，那么只能是下面两种状态：



同理，右倾红黑树也是一样的道理，即红色节点向右倾斜，它的红色节点只能是下面这两种状态：



AA树，是指红黑树中所有的红色节点必须只能是右节点，左子结点一律不允许是红色子结点，所以，AA树中红色子结点只能是一种状态。其实AA树也可以看作是2-3树演化而来。下去以后可以画图体验一下。

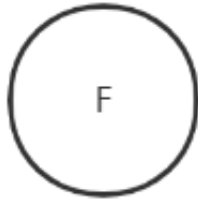
那么对于既有左边的红色子结点，又有右边的红色子结点，就是一颗经典的，普通红黑树了。

## 红黑树的插入

对于红黑树的插入，我们需要先约定一个规则，插入的结点必须是红色，但如果是根结点，就把它涂成黑色。

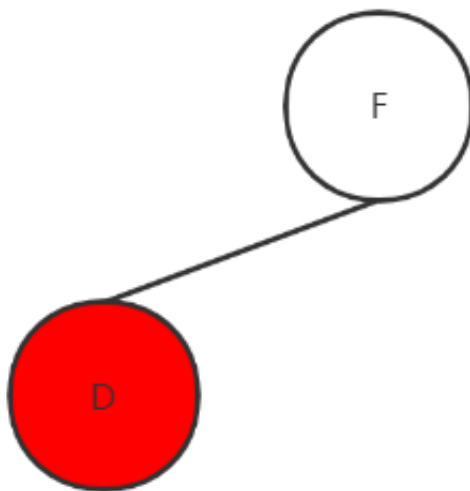
有了这个约定，我们就可以使用图示来慢慢拆解红黑树（左倾举例）插入元素的过程。

1、插入元素F，因为是根结点，所以直接就是黑色。

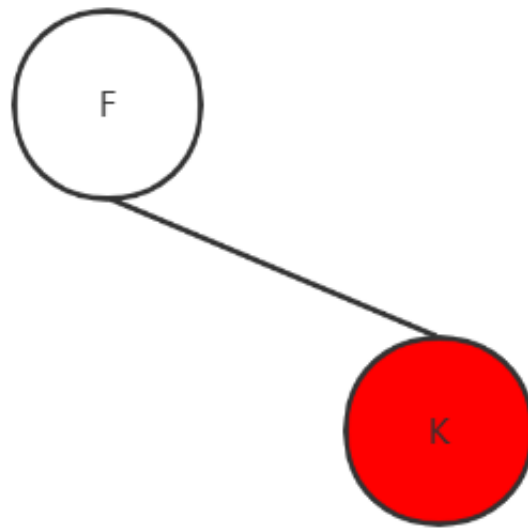


2、插入第二个元素，这里就要分情况，比F大，比F小，

1、假设插入元素D，那么，它比F小，就成为F的左子结点，此时D为红色左子结点，不需要在平衡。



2、假设插入的元素为K，比F大，成为F的右子结点。

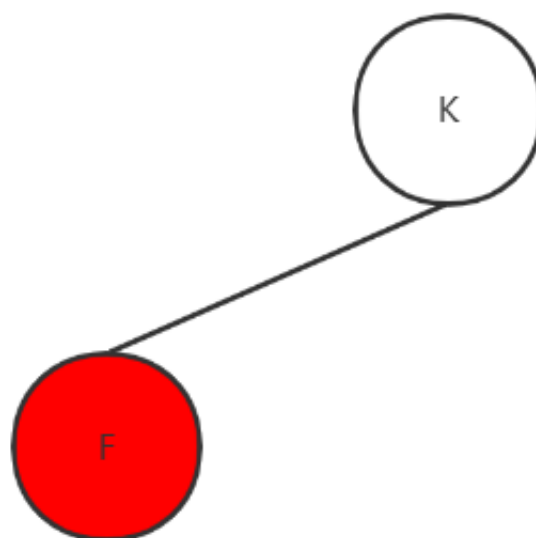


此时，K为红色右子结点，不符合左倾红黑树的规则，所以需要在平衡，那么如何再平衡呢？

回归红黑树的本质，2-3-4树，上面包含F和K两个元素的红黑树变成2-3-4树就变成了



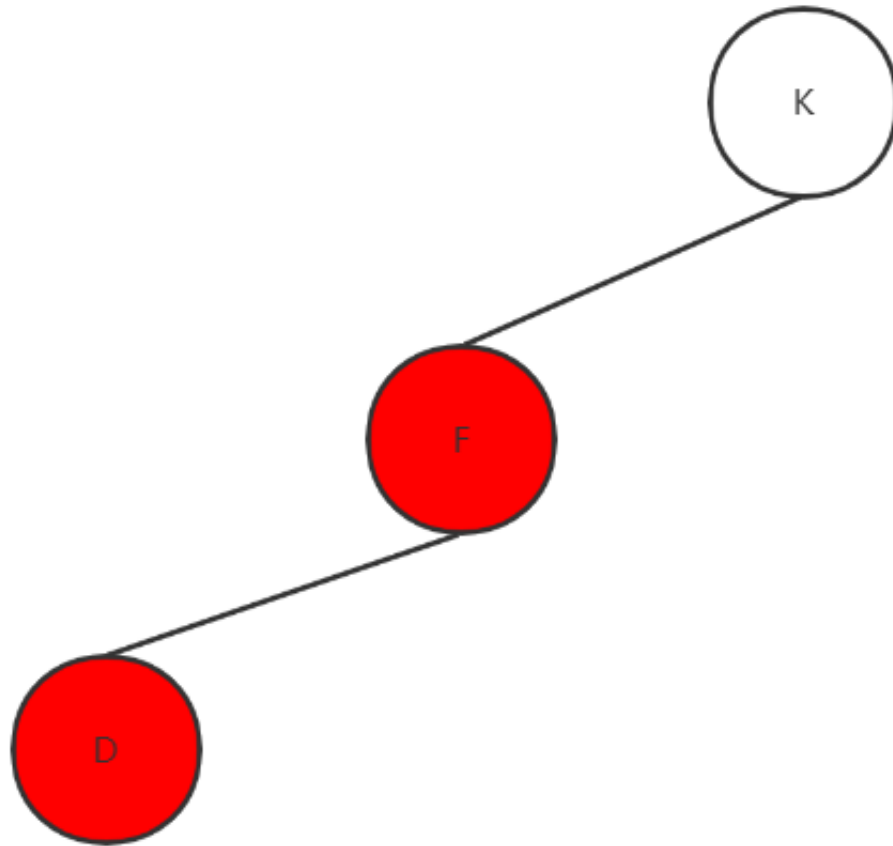
再将这个2-3-4树转换成左倾红黑树就变成了



所以，你看，结合2-3-4树来理解红黑树是不是就特别简单了，对于2-3-4树就是一个普通的3节点，而对于红黑树相当于插入一个右子节点，再做一次左旋变色即可。

3、插入第三个元素 这时候会有三种情况

(1) 我们继续插入元素D，假设插入的元素比F小，所以会成为F的左子结点。

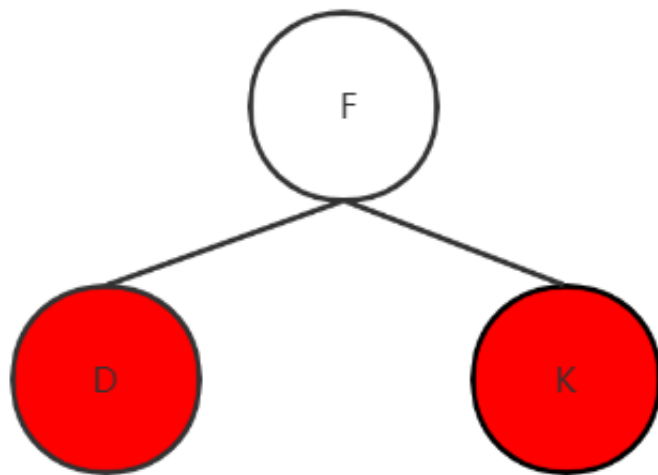


左子结点显然不符合红黑树的定义，所以需要再平衡，那么如何来平衡呢？

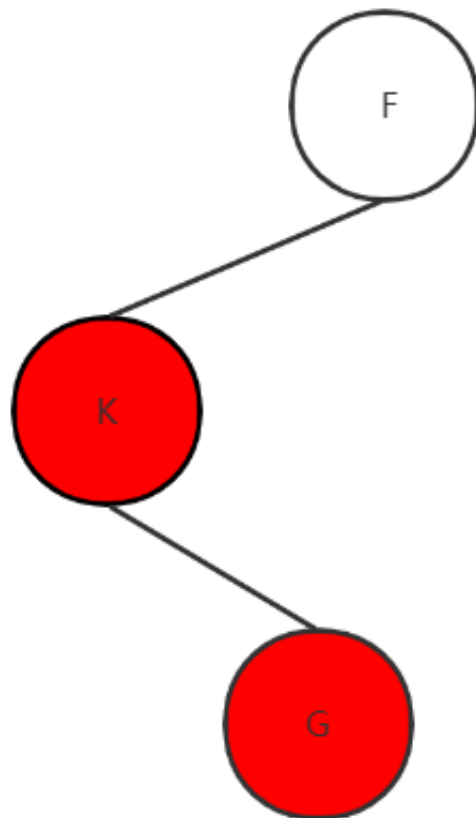
如果是2-3-4树，怎么平衡呢？



对于2-3-4树来说，就是形成一个4结点，对于红黑树来说，相当于右旋。

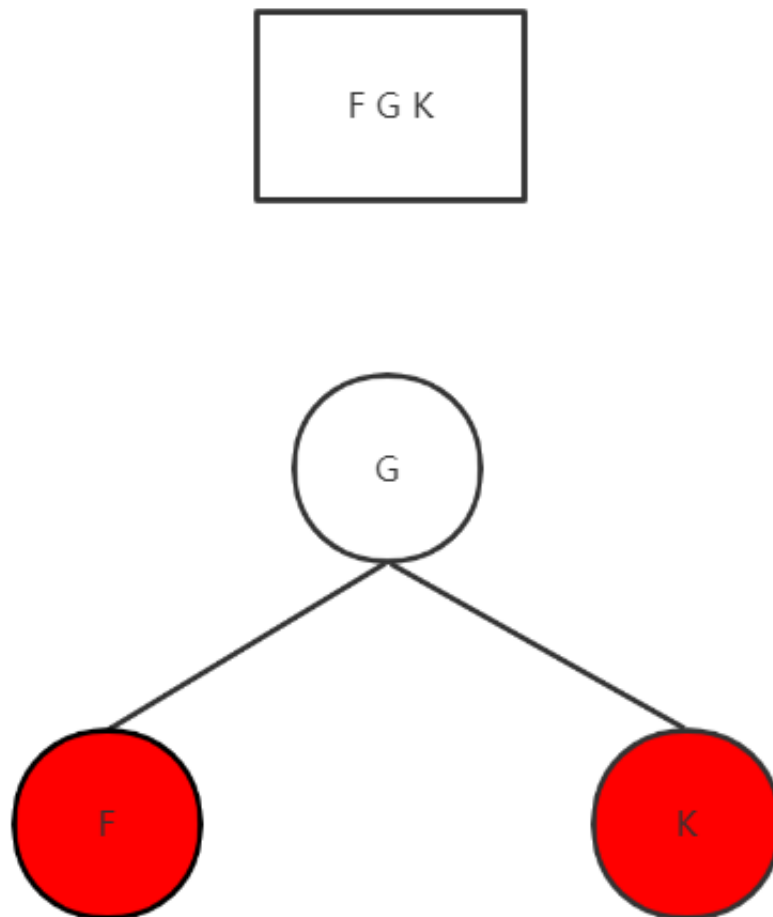


(2) 假设插入的元素G，比F大，比K小，这时候会变成F的右子结点。



显然，他也不符合红黑树的定义，所以，需要再平衡：

插入元素G，对于2-3-4树，只是形成一个普通的4节点，而对于红黑树，需要先以F左旋，变成与情况（1）相同的状态，再以G右旋，然后变色，最终再平衡成红黑树。

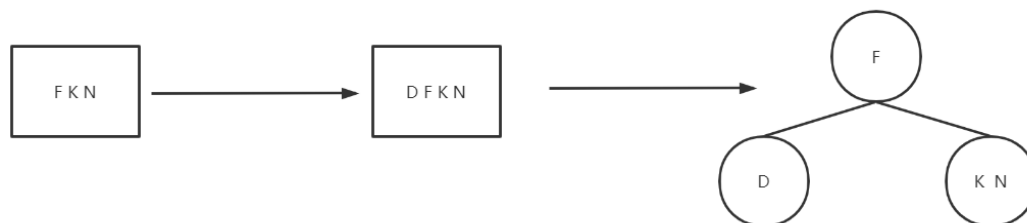


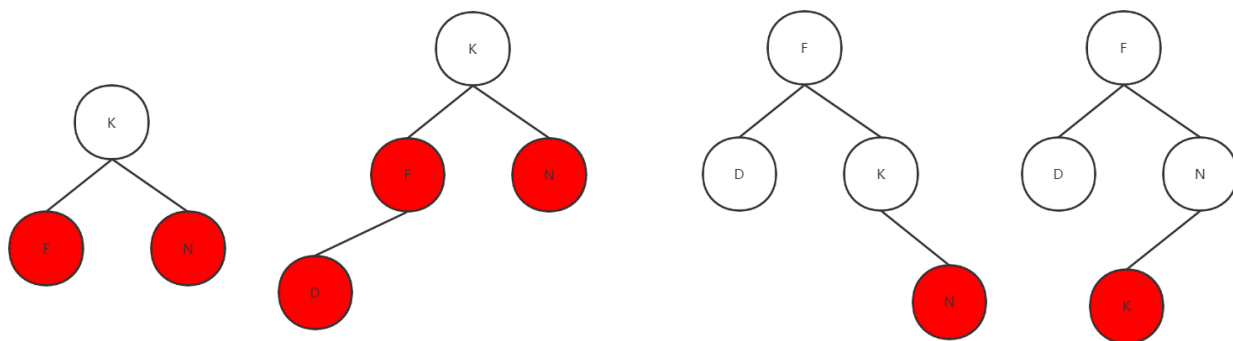
(3) 假设插入的元素为N，它比K大，所以会成为K的右子结点，此时，正好符合红黑树的定义，不需要再平衡了

通过上面的分析，连续插入三个元素，可以看到，对于2-3-4，都是形成一个4节点，而对于红黑树，最终都变成了上面的情况。

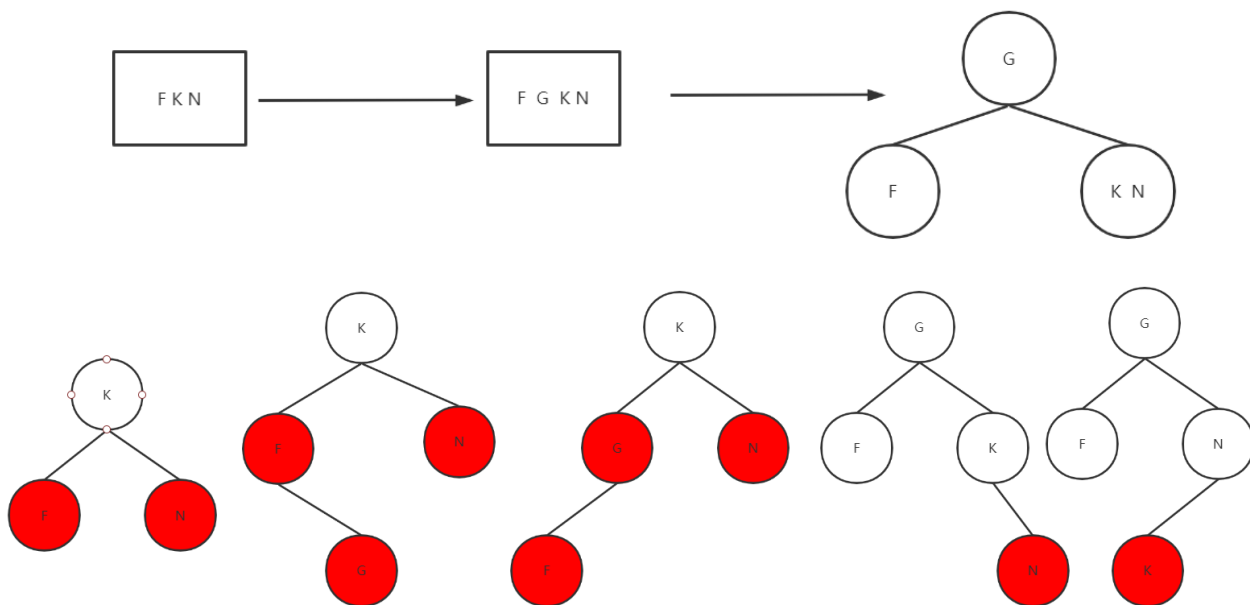
#### 4、插入第四个元素

我们以FKN这颗红黑树为例，插入第四个元素，可能会出现四种情况。我们直接上图  
假设插入D是F的左子结点

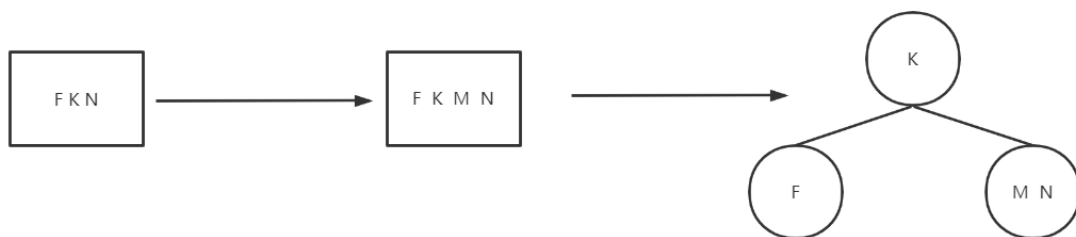


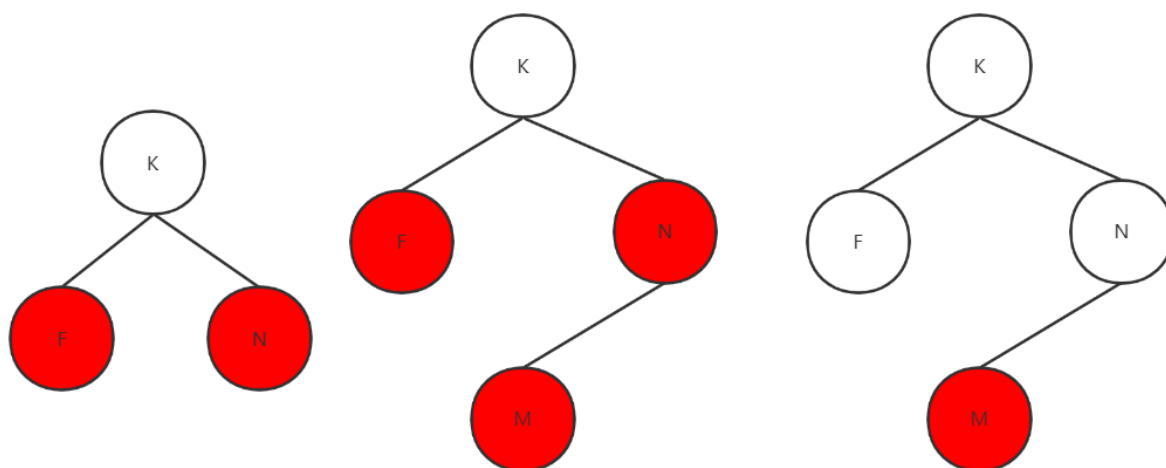


假设插入一个G，其为F的右子结点

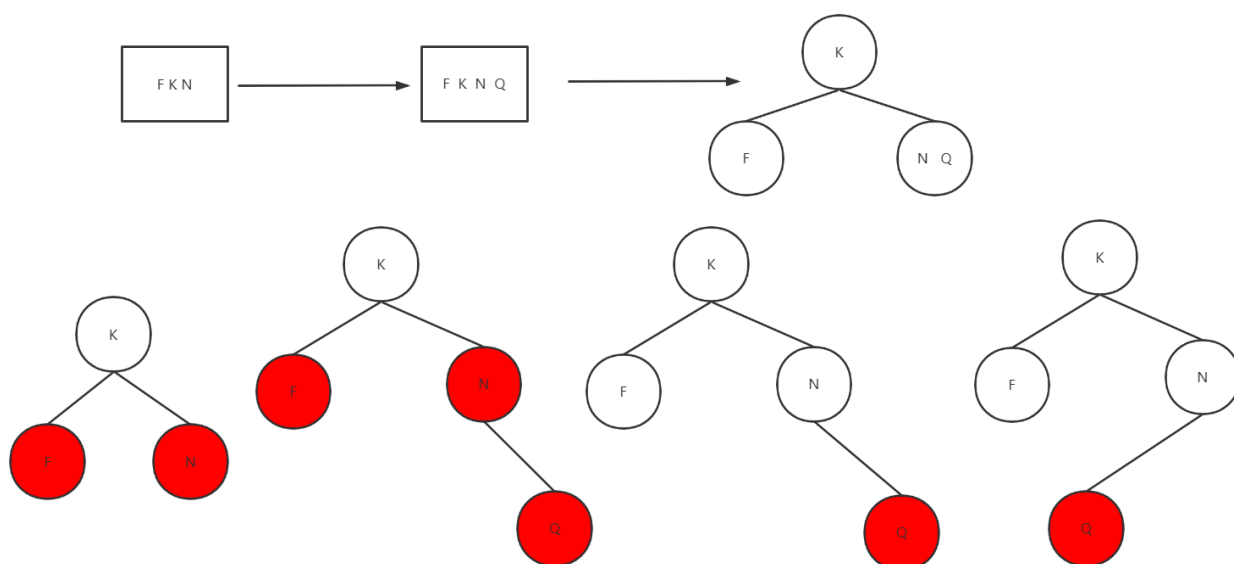


假设插入的是N的左子结点





假设插入的Q，为N的右子结点



好了，插入四个元素的各种情况到此结束，可以看到，插入第四个元素时，对于2-3-4树，会形成一个5节点，然后再分裂，而对于红黑树，要经过一系列的左旋、右旋、变色，最终转变成跟2-3-4树对应的形态，是不是很好玩儿～